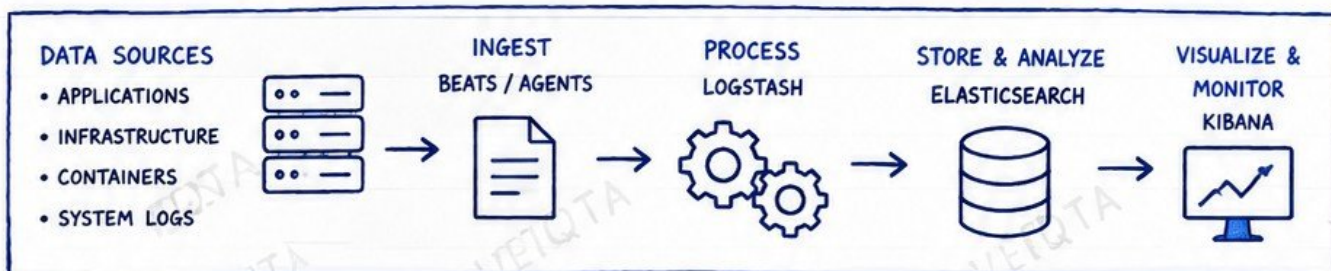


ELK STACK

PRODUCTION HANDBOOK

FROM LOG INGESTION TO
PRODUCTION-SCALE OBSERVABILITY



WHAT YOU WILL LEARN

- ✓ Design scalable logging pipelines
- ✓ Configure and optimize Elasticsearch
- ✓ Parse, enrich, and transform logs with Logstash
- ✓ Collect logs across environments with Beats
- ✓ Build powerful searches and dashboards with Kibana
- ✓ Secure, monitor, and operate ELK in production
- ✓ Troubleshoot issues and recover from failures
- ✓ Real-world architectures and production lessons

PRODUCTION FOCUS

- ★ REAL-WORLD ARCHITECTURES
- ★ PERFORMANCE & SCALING
- ★ SECURITY & RELIABILITY
- ★ TROUBLESHOOTING
- ★ BEST PRACTICES



BUILD OBSERVABILITY THAT SCALES.
OPERATE WITH CONFIDENCE.

VERI QTA

ENGINEER • OBSERVE • UNDERSTAND

1. ELK STACK ARCHITECTURE IN PRODUCTION

WHAT ELK ACTUALLY SOLVES

- COLLECT LOGS FROM ANYWHERE
- CENTRALIZE AND STORE AT SCALE
- SEARCH, ANALYZE, AND CORRELATE
- BUILD DASHBOARDS AND ALERTS
- IMPROVE OBSERVABILITY & MTTR

WHERE BEATS & ELASTIC AGENT FIT

BEATS / ELASTIC AGENT



- LIGHTWEIGHT AGENTS
- COLLECT LOGS & METRICS
- SHIP DATA RELIABLY
- MINIMAL RESOURCE USAGE

LOGSTASH (OPTIONAL)



- PARSE, ENRICH, FILTER
- TRANSFORM & ROUTE
- HANDLE COMPLEX LOGIC
- BUFFER & RELIABILITY

THE ELK COMPONENTS



ELASTICSEARCH

DISTRIBUTED SEARCH AND STORAGE ENGINE



LOGSTASH

PROCESS, ENRICH, TRANSFORM, AND ROUTE LOGS



KIBANA

VISUALIZE, ANALYZE, AND CREATE INSIGHTS

ELASTICSEARCH



- INDEX & STORE
- SEARCH & AGGREGATE
- REPLICATE & SCALE
- HIGH AVAILABILITY

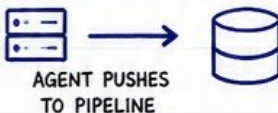
KIBANA



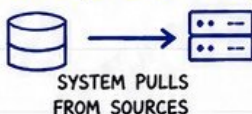
- VISUALIZE DATA
- DASHBOARDS
- ALERTS
- REPORTING

COLLECTION PATTERNS

PUSH (COMMON)

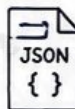


PULL (LESS COMMON)



LOG TYPES

STRUCTURED LOGS



- JSON / KEY-VALUE
- EASY TO PARSE
- POWERFUL QUERIES
- RECOMMENDED

UNSTRUCTURED LOGS

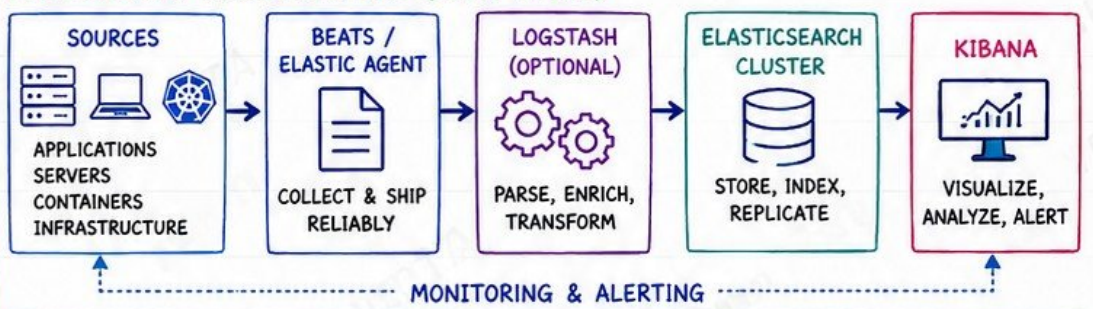


- FREE FORM TEXT
- HARDER TO PARSE
- REQUIRES PATTERNS
- MORE PROCESSING

WHERE FAILURES HAPPEN

- ⚠ AT THE SOURCE (AGENT DOWN)
- ⚠ NETWORK / CONNECTIVITY
- ⚠ LOGSTASH PIPELINE ISSUES
- ⚠ ELASTICSEARCH REJECTIONS
- ⚠ DISK / HEAP PRESSURE
- ⚠ SHARD ALLOCATION FAILURES
- ⚠ KIBANA / VISUALIZATION ISSUES

PRODUCTION ARCHITECTURE (HIGH LEVEL)



PRODUCTION NOTE

DESIGN THE PIPELINE AROUND FAILURE AND VOLUME, NOT DASHBOARDS. DASHBOARDS COME LAST.



ELK IS MORE THAN A LOG VIEWER.
IT IS AN OBSERVABILITY PLATFORM FOR PRODUCTION SYSTEMS.

2. DESIGNING A PRODUCTION LOGGING PIPELINE

Decide how logs should travel through your infrastructure.

1. APPLICATION LOG SOURCES



STDOUT AND STDERR
Application output



FILES AND SYSTEM LOGS
Log files, logrotate, syslog



KUBERNETES LOGS
Container logs, events, controller logs



INFRASTRUCTURE LOGS
OS, network, DB, security, load balancers

2. INGESTION ARCHITECTURE OPTIONS

A

DIRECT INGESTION

Agent sends directly to Elasticsearch



B

VIA LOGSTASH

Agent sends to Logstash for processing



C

WITH QUEUE BUFFER

Agent sends to Kafka for buffering



D

HYBRID (HIGH SCALE)

Agent → Kafka → Logstash → Elasticsearch



3. KEY CONSIDERATIONS



BACKPRESSURE

- Handle slow downstream without data loss



NETWORK FAILURE BEHAVIOR

- Retry, buffer, or drop?
- Know your failure mode



DELIVERY GUARANTEES

- At-most-once
- At-least-once
- Exactly-once (rare)



PIPELINE BOTTLENECKS

- Network, parsing, queues,
- CPU, disk, Elasticsearch capacity

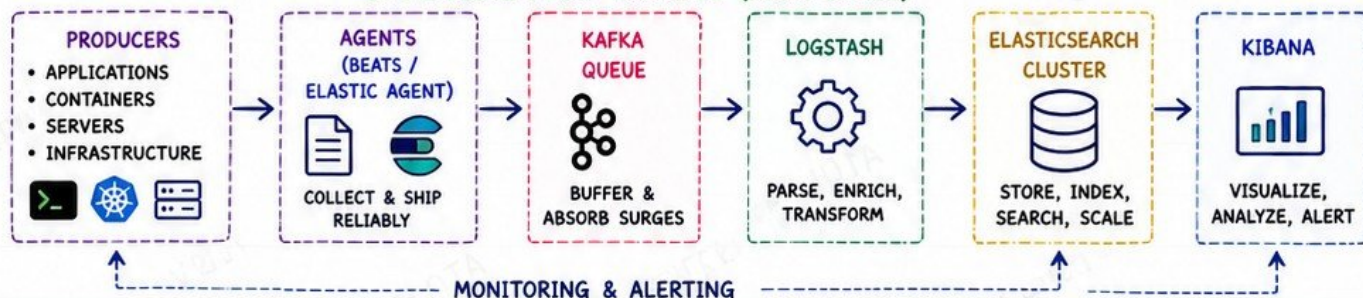
4. COMPARISON AT A GLANCE

OPTION	COMPLEXITY	RELIABILITY	BEST FOR
A	LOW	MEDIUM	SMALL ENVIRONMENTS
B	MEDIUM	HIGH	PROCESSING NEEDS
C	MEDIUM	HIGH	BURSTY LOADS BUFFERING
D	HIGH	VERY HIGH	LARGE SCALE SYSTEMS

5. CHOOSING THE RIGHT ARCHITECTURE

- ☒ START WITH YOUR LOG VOLUME
- ☒ CONSIDER TOLERABLE DATA LOSS
- ☒ EVALUATE PROCESSING NEEDS
- ☒ PLAN FOR GROWTH AND FAILURE
- ☒ KEEP ELASTICSEARCH PROTECTED

6. PIPELINE FLOW EXAMPLE (HIGH SCALE)



ENTERPRISE ADVICE: DECOUPLE PRODUCERS FROM ELASTICSEARCH AT LARGE SCALE. BUFFER, ABSORB, AND PROTECT YOUR DATA PIPELINE.

3. HOW ELASTICSEARCH STORES AND SEARCHES LOGS

UNDERSTAND WHAT HAPPENS AFTER INGESTION.

1. DOCUMENTS (JSON REPRESENTATION)

```
{
  "@timestamp": "2025-05-24T10:15:30Z",
  "level": "ERROR",
  "message": "Failed to connect to DB",
  "user.id": "u12345",
  "latency_ms": 1250,
  "source.ip": "10.0.1.25",
  "service": "payment-api"
}
```

★ EVERY LOG EVENT IS STORED AS A JSON DOCUMENT.

2. INDICES

INDEX IS A COLLECTION OF DOCUMENTS.



TIMESTAMPED INDICES HELP WITH SCALING AND RETENTION.

3. FIELDS & MAPPING

FIELD	TYPE	DESCRIPTION
@timestamp	date	When the event occurred
level	keyword	Exact value (INFO, ERROR)
message	text	Full-text search
user.id	keyword	Exact match / aggregations
latency_ms	long	Numeric value
source.ip	ip	IP address type
service	keyword	Exact value

MAPPING DEFINES HOW FIELDS ARE INDEXED AND SEARCHED.

4. INVERTED INDEX (THE MAGIC)

Original Document

```
user: alice
action: login
status: success
ip: 10.0.1.5
...
```

Inverted Index

```
user → [alice] → [Doc 1]
action → [login] → [Doc 1]
status → [success] → [Doc 1]
ip → [10.0.1.5] → [Doc 1]
```

ELASTICSEARCH BUILDS AN INVERTED INDEX SO IT CAN FIND DOCUMENTS FAST.

5. TEXT ANALYSIS & TOKENS

Text fields are analyzed (broken into tokens).

Example: "Failed to connect to DB"



Tokens:

failed to connect db

6. TEXT vs KEYWORD

text

- Analyzed
- Tokenized
- Full-text search
- Not for exact match

keyword

- Not analyzed
- Exact value
- Good for filtering, aggregations, sorting

7. SEARCH TYPES

FULL-TEXT SEARCH
(text fields)

failed connect

Matches documents containing these terms.

EXACT MATCH SEARCH
(keyword fields)

level:ERROR

Matches exact value only.

8. DYNAMIC MAPPING



Elasticsearch can automatically detect new fields and assign a type.

Great for agility, but dangerous when field names or types vary.

9. TIMESTAMP, NUMERIC & IP FIELDS



TIMESTAMP (date):
Used for time filtering, sorting, ILM, rollovers.

123

NUMERIC (long, double):
For metrics, aggregations, range queries.



IP (ip):

Special type for IP range queries.



10. MAPPING CONFLICTS

When the same field has different types (e.g., text vs keyword), indexing fails or data becomes inconsistent.



PLAN MAPPINGS EARLY. STAY CONSISTENT.

11. WHY FIELD DESIGN MATTERS



- Affects search speed
- Impacts storage
- Controls aggregations
- Prevents mapping explosion
- Reduces cluster load



SENIOR ENGINEER TIP: TREAT MAPPINGS AS PART OF YOUR DATA ARCHITECTURE.
GOOD FIELD DESIGN TODAY PREVENTS PAIN AND SLOW PERFORMANCE TOMORROW.

4. ELASTICSEARCH NODES, CLUSTERS, SHARDS, AND REPLICAS

UNDERSTAND THE DISTRIBUTED ARCHITECTURE BEHIND THE SEARCH ENGINE.

1. CLUSTER

- A cluster is a collection of nodes working together as one search engine.
- All data is split into shards and distributed across nodes.

ELASTICSEARCH CLUSTER



2. NODES

- A node is a single server in the cluster.
- Nodes can have multiple roles.



NODE
CPU
RAM
DISK
NETWORK

3. NODE ROLES

ROLE	RESPONSIBILITY
 MASTER-ELIGIBLE NODE	- Manages cluster state - Creates / deletes indices - Decides shard allocation
 DATA NODE	- Stores data (shards) - Handles search & indexing
 INGEST NODE	- Pre-processes documents - Runs ingest pipelines
 COORDINATING NODE	- Routes requests - Aggregates responses

4. SHARDS & REPLICAS

PRIMARY SHARD

- Holds the actual data.



PRIMARY

REPLICA SHARD

- Copy of primary shard for high availability.



REPLICA

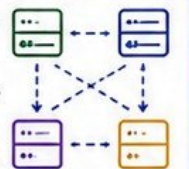
SYNC / ASYNC
REPLICATION

BENEFITS

- High availability
- Fault tolerance
- Read scalability
- Data durability

5. SHARD ALLOCATION

- Shards are distributed across nodes based on cluster rules.
- Elasticsearch tries to balance shards to optimize resource usage.



6. CLUSTER STATE

- Contains metadata about the cluster.
- Includes indices, mappings, shards, nodes, settings.
- Stored and updated by master-eligible nodes.



7. NODE DISCOVERY

- Nodes must find each other to join the cluster.
- Uses discovery settings (seeds, unicast, DNS, etc.).



8. QUORUM CONCEPTS

- Master decisions require a quorum of master-eligible nodes.
- Prevents split-brain and ensures cluster consistency.



9. NODE FAILURE BEHAVIOR

- Shards on failed node are reallocated.
- Replicas keep the cluster available.
- If too many masters fail, cluster becomes unavailable.



REPLICA TAKES OVER



10. WHY TOO MANY SHARDS BECOME EXPENSIVE

- More shards = more heap usage (metadata).
- Slower cluster state updates.
- More file handles & I/O.
- Longer recovery times.
- Higher CPU overhead.



KEY TAKEAWAYS

- Design for balance.
- Use replicas for reliability.
- Monitor shard count.
- Plan for failure.



PRODUCTION FAILURE

OVERSHARDING CAN DESTROY AN OTHERWISE HEALTHY CLUSTER.

KEEP SHARDS RIGHT-SIZED. LESS IS OFTEN MORE.

5. INDEX DESIGN FOR PRODUCTION LOGS

DESIGN INDICES FOR SCALE RATHER THAN CONVENIENCE.

1. INDEX NAMING

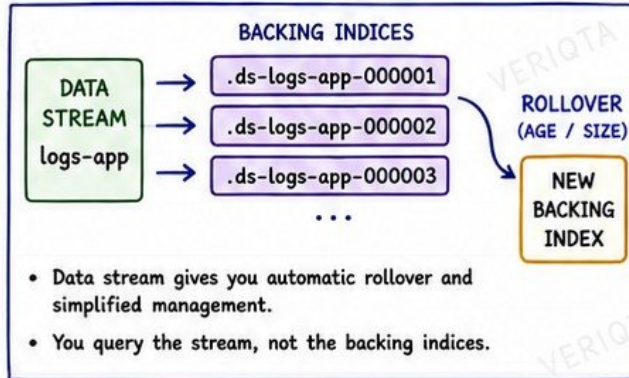
GOOD PATTERN (TIME-SERIES)

logs-<dataset>-YYYY.MM.DD

Example:
logs-app-2025.05.24

- ✓ Predictable
- ✓ Easy to manage
- ✓ Works with ILM & data streams

2. TIME-SERIES WITH DATA STREAMS



3. TEMPLATES

INDEX TEMPLATE



- Defines mappings
- Defines settings

COMPONENT TEMPLATES



- Reusable building blocks
- Keep templates DRY

4. MAPPINGS (SCHEMA OF YOUR DATA)

```
{
  "properties": {
    "@timestamp": { "type": "date" },
    "level": { "type": "keyword" },
    "message": { "type": "text" },
    "user.id": { "type": "keyword" },
    "latency.ms": { "type": "long" },
    "source.ip": { "type": "ip" }
  }
}
```

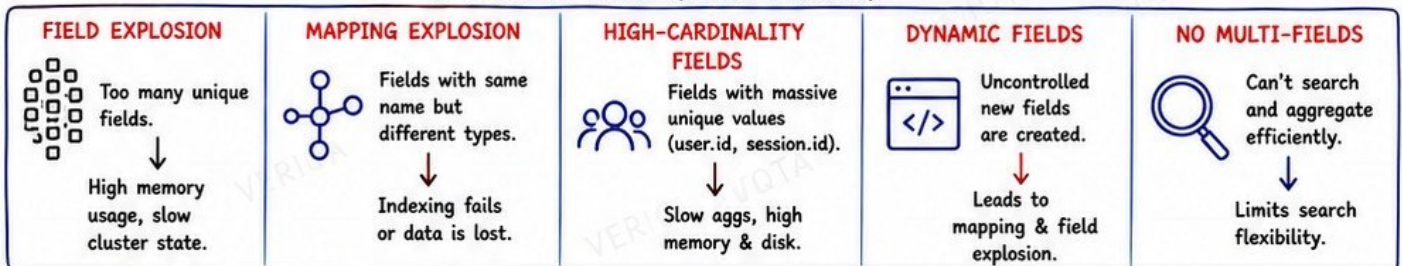
- ✓ Define field types explicitly.
- ✓ Prevents mapping conflicts.
- ✓ Improves search performance.
- ✓ Reduces index size.

5. SETTINGS (PERFORMANCE & BEHAVIOR)

```
{
  "settings": {
    "number_of_shards": 3,
    "number_of_replicas": 1,
    "refresh_interval": "30s",
    "codec": "best_compression",
    "index.lifecycle.name": "logs-ilm"
  }
}
```

- ✓ Control shard count
- ✓ Replica strategy
- ✓ Refresh interval
- ✓ Compression
- ✓ ILM policy

6. WHAT TO AVOID (SCALE KILLERS)



7. MULTI-FIELDS (SEARCH + AGGREGATE)

```
"message": {
  "type": "text",
  "fields": {
    "keyword": { "type": "keyword", "ignore_above": 256 }
  }
}
```

- ✓ Use text for full-text search.
- ✓ Use keyword for exact match and aggregations.
- ✓ Best of both worlds.

8. INDEXING STRATEGY



- ✓ Use bulk API.
- ✓ Batch requests.
- ✓ Handle retries.
- ✓ Monitor rejections.

9. SEARCH IMPLICATIONS

- Well-designed mappings = faster queries.
- Use keyword for filters & aggregations.
- Avoid leading wildcards.
- Minimize high-cardinality aggregations.

10. STORAGE IMPLICATIONS

- Smaller mappings = smaller index.
- Compression reduces disk usage.
- Avoid unnecessary fields.
- ILM tiers reduce storage cost.



PRODUCTION NOTE

BAD SCHEMA DECISIONS
BECOME EXPENSIVE
AT TERABYTE SCALE.



DESIGN YOUR INDICES LIKE A DATABASE.
YOU CAN'T FIX A BAD SCHEMA AFTER THE DATA HAS GROWN.

6. INDEX LIFECYCLE MANAGEMENT AND DATA TIERS

CONTROL HOW LOG DATA AGES.

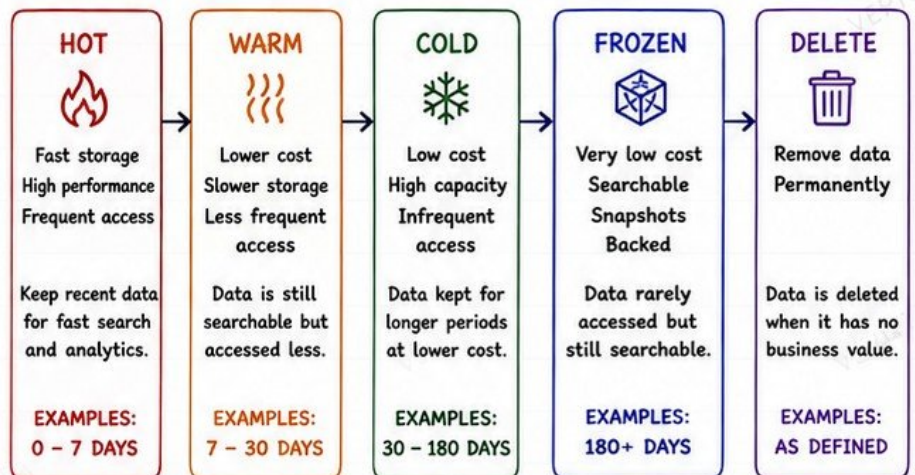
1. WHY UNLIMITED RETENTION FAILS

- ⊗ Storage grows without limit
- ⊗ Search performance degrades
- ⊗ Backups take too long
- ⊗ Cluster becomes unstable
- ⊗ Costs spiral out of control
- ⊗ You keep data with no value



**MORE DATA ≠
MORE VALUE**

2. ILM LIFECYCLE PHASES



3. ROLLOVER: KEEP INDICES HEALTHY AND MANAGEABLE



Rollover creates a new index when conditions are met.

**KEEPS SHARDS SMALL
AND PERFORMANCE HIGH.**

4. ROLLOVER CONDITIONS

AGE-BASED



Rollover when an index reaches a certain age.

Example:
max_age: 7d

SIZE-BASED



Rollover when an index reaches a certain size.

Example:
max_size: 50gb

YOU CAN COMBINE AGE + SIZE CONDITIONS.

5. DATA TIERS (UNDER THE HOOD)



HOT TIER → SSD, High Performance, Expensive



WARM TIER → HDD / Warm Storage, Lower Cost



COLD TIER → Low Cost Storage, High Capacity



FROZEN TIER → Searchable Snapshots, Cheapest

ILM MOVES DATA BETWEEN TIERS AUTOMATICALLY.

6. RETENTION POLICIES

- ✓ Define how long data stays in each phase.
- ✓ Example: Hot 7d → Warm 30d → Cold 180d → Delete
- ✓ Align retention with business and compliance needs.
- ✓ Different data types can have different policies.

7. STORAGE COST vs SEARCH PERFORMANCE



MORE PERFORMANCE

- Keep in Hot tier
- Higher cost
- Faster searches



LOWER COST



- Move to Warm / Cold / Frozen
- Slower searches
- Much cheaper

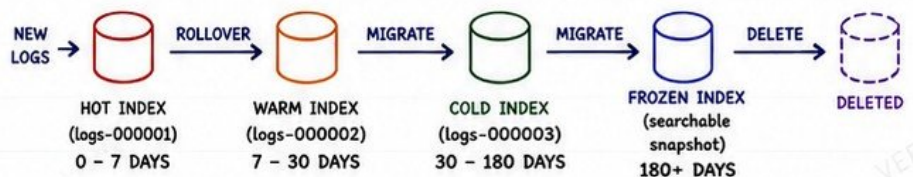
BALANCE COST AND PERFORMANCE BASED ON DATA VALUE.

8. COMPLIANCE CONSIDERATIONS



- Legal and regulatory requirements.
- Audit and eDiscovery needs.
- Immutable / WORM storage options.
- Retention must be provable and enforceable.

9. ILM FLOW EXAMPLE



**ENTERPRISE ADVICE:
RETENTION SHOULD FOLLOW BUSINESS VALUE, NOT HABIT.**

7. LOGSTASH PIPELINE ARCHITECTURE

UNDERSTAND LOGSTASH BEYOND BASIC INPUT, FILTER, OUTPUT EXAMPLES.

1. CORE BUILDING BLOCKS



INPUTS

Receive data from multiple sources.



FILTERS

Process, parse, enrich, and transform events.



OUTPUTS

Send processed events to one or more destinations.



EVENTS

A unit of data flowing through the pipeline (JSON structure).



CODECS

Encode or decode data to/from wire formats.

2. PIPELINE WORKERS & BATCHING



PIPELINE WORKERS

- Multiple threads process events in parallel.
- Increases throughput and CPU utilization.



BATCH PROCESSING

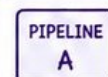
- Events are collected into batches.
- Batches improve efficiency and reduce overhead.



TUNABLE SETTINGS

- pipeline.workers
- pipeline.batch.size
- pipeline.batch.delay

3. MULTIPLE PIPELINES



- Separate pipelines isolate workloads.
- Prevents one pipeline from blocking others.
- Each pipeline has its own config and settings.

4. CONDITIONAL PROCESSING



IF / ELSE IF / ELSE

- Route events based on fields, values, tags, or metadata.
- Process only what you need.
- Drop what you don't need.

5. PIPELINE CONFIGURATION

```
input {
  beats { port => 5044 }
}
filter {
  if [type] == "nginx" {
    grok { match => { "message" => "%{COMBINEDAPACHELOG}" } }
  }
  mutate { add_field => { "env" => "prod" } }
}
output {
  elasticsearch { hosts => ["es1:9200", "es2:9200"] }
  if "_grokparsefailure" in [tags] {
    file { path => "/var/log/logstash/dlq-failures.log" }
  }
}
```

Pipelines are defined in .conf files. Keep them modular and readable.

6. RELIABILITY COMPONENTS



PERSISTENT QUEUES

- Stores events on disk before they are processed.
- Survives Logstash restarts.
- Protects against temporary downstream failures.



DEAD LETTER QUEUES (DLQ)

- Captures events that cannot be processed or indexed.
- Prevents data loss.
- Inspect, fix, and reprocess later.

7. BACKPRESSURE & FLOW CONTROL



BACKPRESSURE HAPPENS WHEN

- Outputs are slow or unavailable.
- Queues fill up.
- Resources (CPU, memory, disk) are saturated.



LOGSTASH REACTION

- Accepts data up to the queue limit.
- Slows inputs to prevent overwhelm.
- Applies backpressure to upstream sources (when supported).



FLOW CONTROL GOAL

- Keep the pipeline stable, protect data, and avoid data loss.

8. PIPELINE BOTTLENECKS

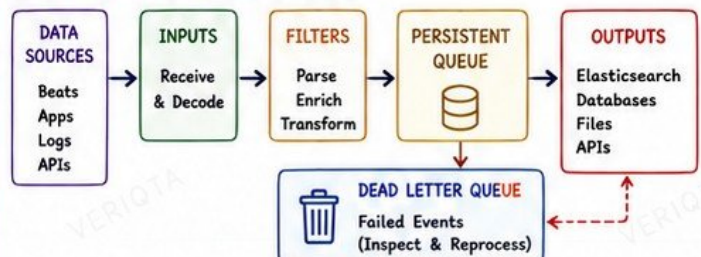
- Slow outputs (ES, DB, HTTP, etc.)
- Complex or slow filters (regex, grok)
- Too small batch size
- Too few pipeline workers
- High field cardinality in outputs
- Insufficient CPU, memory or disk I/O
- Network latency or packet loss

9. FAILURE HANDLING



- Retries for recoverable errors.
- Failed events go to DLQ.
- Persistent queues protect during crashes.
- Monitor pipeline health continuously.
- Alert on failures and queue growth.

10. PRODUCTION FLOW DIAGRAM



ENTERPRISE ADVICE: RETENTION SHOULD FOLLOW BUSINESS VALUE, NOT HABIT.

8. PARSING AND TRANSFORMING LOGS WITH LOGSTASH

TURN RAW EVENTS INTO USEFUL STRUCTURED DATA.

1. CORE PARSING FILTERS



GROK

Extract fields using regex patterns.
Great for complex and inconsistent logs.

Example:

```
{IP:client} - - [%{HTTPDATE:timestamp}\}
```



DISSECT

Fast, reliable parsing for well-structured logs.

Example:

```
{client} - - [%{timestamp}] "%{verb} %{url}"
```



JSON PARSING

Parse JSON logs into structured fields.

Example:

```
{ "level": "info", "msg": "ok" }
```



DATE PARSING

Convert timestamp strings to proper @timestamp.

Example:

```
yyyy-MM-dd HH:mm:ss.SSS
```

4. DROPPING UNWANTED EVENTS



Drop noisy or useless events early.

Example:

```
if [log_level] == "debug" {
  drop { }
}
```

5. TAGS & METADATA



TAGS

- Mark events for routing, filtering, or alerting.
- Example: `add_tag => ["web", "nginx", "prod"]`



METADATA

- Add contextual info not from the original log.
- Example: `add_field => { "env" => "prod" }`

2. ENRICHMENT & TRANSFORMATION



MUTATE

Add, rename, remove, convert, lowercase, uppercase, strip, replace fields.



GEOIP

Add geo-location based on IP address.
Adds city, region, country, lat, lon.



USER-AGENT ENRICHMENT

Extract browser, OS, device, and engine from user-agent.



FIELD NORMALIZATION

Standardize field names and values.
Keep data consistent across sources.

3. CONDITIONAL FILTERING



Process events only when conditions match.

Example:

```
if [log_type] == "nginx" {
  grok { match => { "message" => "%{COMBINEDAPACHELOG}" } }
} else if [log_type] == "app" {
  json { source => "message" }
}
```

6. PARSING FAILURES



- Failed grok adds `_grokparsefailure` tag.
- JSON errors go to `_jsonparsefailure`.
- Handle failures explicitly, don't ignore them.
- Send failures to DLQ or separate index.

7. EXPENSIVE REGEX PATTERNS



- Catastrophic backtracking can crush CPU.
- Avoid nested quantifiers like `(.)*+`
- Use DISSECT when possible.
- Test patterns with real data volume.

PRODUCTION FLOW DIAGRAM

RAW LOG



INPUT



FILTERS (PARSE & TRANSFORM)



- GROK / DISSECT
- JSON / DATE
- MUTATE / GEOIP
- CONDITIONALS
- NORMALIZATION
- DROP / TAG / META

SUCCESS EVENTS



CLEAN & STRUCTURED

OUTPUT



ELASTICSEARCH OR OTHERS

FAILED EVENTS



DLQ OR FAILURE INDEX



DEBUGGING INSIGHT: PARSING SHOULD FAIL VISIBLY, NOT SILENTLY.
IF YOU CAN'T SEE THE FAILURE, YOU CAN'T FIX THE DATA.

9. BEATS AND ELASTIC AGENT

COLLECT TELEMETRY SAFELY FROM DISTRIBUTED SYSTEMS.

1. WHY LIGHTWEIGHT COLLECTORS EXIST



- Run on edge close to data sources.
- Minimal CPU, memory and disk usage.
- Reliable delivery even over unstable networks.
- Offload shipping, parsing and enrichment from sources.
- Scale horizontally with zero impact on applications.

3. HOW BEATS / AGENT WORKS



4. KEY CONCEPTS

	INPUTS	Define what to collect (log files, metrics, HTTP, etc).
	HARVESTERS	Read files line by line and send only new content.
	REGISTRIES	Track file states (inode, offset) to avoid duplicates.
	MULTILINE LOGS	Combine stack traces and multi-line messages into single events.
	LOG ROTATION	Detect renamed or rotated files and continue without data loss.

5. OUTPUT CONFIGURATION



- Elasticsearch (direct)
- Logstash (central processing)
- Kafka / Redis / S3 / Others
- Load balancing and failover supported.
- TLS, authentication, and API keys.

2. BEATS FAMILY



FILEBEAT

Collect and ship log files. Best for application, system, and container logs.



METRICBEAT

Collect system and service metrics.



HEARTBEAT

Check the availability of services and endpoints.



AUDITBEAT

Collect audit data from systems and apps.



ELASTIC AGENT

Unified agent for logs, metrics, APM, security, and more.

6. BUFFERING & RELIABILITY



- Events are queued in memory and on disk.
- Survives temporary network or output failures.
- Backoff and retry with exponential delay.
- At-least-once delivery guarantees.

7. AGENT RESOURCE USAGE



- Lightweight by design.
- Typical usage: low CPU (<2%), memory (<100MB).
- Monitor on high-volume hosts.

8. LOST-EVENT SCENARIOS



- Disk full on agent (registry or queue).
- File deleted before harvester reads it.
- Output unreachable beyond retry limits.
- Misconfigured ignores or includes.
- System crash with disabled disk queue.

9. FILEBEAT VS LOGSTASH RESPONSIBILITIES

FILEBEAT (EDGE)		LOGSTASH (CENTRAL)
Collect & ship logs	→	Parse, enrich, transform
Lightweight agent	→	Heavy processing engine
Low resource usage	→	High resource processing
Resilient shipping	→	Central pipeline control
Minimal parsing	→	Complex parsing & routing



PRODUCTION NOTE: KEEP EDGE COLLECTION LIGHTWEIGHT. COLLECT RELIABLY. TRANSFORM CENTRALLY.

10. KUBERNETES LOGGING WITH ELK

BUILD CENTRALIZED LOGGING FOR CONTAINER PLATFORMS.

1. LOG SOURCES IN KUBERNETES



CONTAINER STDOUT / STDERR

Applications write logs to stdout and stderr.



CONTAINER RUNTIME LOGS

CRI logs from containerd, containerd, or CRI-O.



SYSTEM / NODE LOGS

Kubelet, kube-proxy, system services, audit logs.



APPLICATION FILE LOGS

Apps that write logs to files inside the container.

2. NODE-LEVEL COLLECTION

- Collect logs on each node where pods run.
- Access container logs from `/var/log/containers/`
- Use DaemonSet to run collectors on every node.
- Survives pod restarts and node reboots.

COMMON COLLECTORS



FILEBEAT



ELASTIC AGENT

3. KUBERNETES METADATA ENRICHMENT

Enrich every event with Kubernetes context.



NAMESPACE

Where the workload lives.



POD

Pod name and UID.



CONTAINER

Container name.



NODE

Node name and IP.



LABELS

App, version, team, env, etc.

This metadata is added as fields to improve search and filtering.

4. MULTILINE & LOG HANDLING



MULTILINE LOGS

- Stack traces and multi-line messages must be combined into single events.
- Use multiline patterns in Filebeat/Agent.



LOG ROTATION

- CRI rotates logs automatically.
- Collectors track files using inode + offset (registry).
- Ensures no duplicates or data loss.

5. EPHEMERAL PODS



- Pods can start and disappear in seconds.
- Logs must be shipped quickly off the node.
- DaemonSet collectors capture logs before pod is gone.



BEST PRACTICE

- Do not rely on manual `kubectl logs`. Centralize immediately.

6. KUBERNETES → ELASTICSEARCH FLOW



POD / CONTAINER

App writes logs to stdout/stderr or log files.



NODE FILE SYSTEM

Logs written to `/var/log/containers/`



DAEMONSET COLLECTOR

Filebeat or Elastic Agent collects and enriches logs.



ELASTICSEARCH

Stores logs for search and analysis.



KIBANA

Visualize, explore, and alert on logs.

7. HIGH-CARDINALITY LABEL PROBLEMS



- High-cardinality fields (pod name, UID, request-id) create millions of unique values.
- Increases index size, memory usage, and cost.
- Slows aggregations and dashboards.
- Avoid indexing unnecessary high-cardinality fields.
- Use labels carefully. Not all labels should be indexed.

8. PRODUCTION ARCHITECTURE DIAGRAM



SENIOR ENGINEER TIP:

KUBERNETES METADATA CAN BECOME AN INDEXING LIABILITY.
STORE WHAT YOU NEED. INDEX WHAT YOU USE. IGNORE THE REST.



PRODUCTION NOTE

CENTRALIZE EARLY.
ENRICH SMARTLY.
KEEP LOGS USEFUL,
NOT EXPENSIVE.

11. SEARCHING ELASTICSEARCH LIKE AN ENGINEER

MOVE BEYOND TYPING WORDS INTO KIBANA.

1. QUERY DSL BASICS

Elasticsearch uses JSON-based Query DSL to search with precision.

```
{
  "query": {
    "match": { "message": "error" }
  }
}
```

2. COMMON QUERY CLAUSES

MATCH	Full-text search. Analyzed field.
TERM	Exact match. Not analyzed.
BOOL	Combine multiple conditions.
MUST	Condition must match.
FILTER	Filter results (no scoring).
SHOULD	Optional condition.
MUST_NOT	Exclude matching results.

3. BOOL QUERY EXAMPLE

```
{
  "query": {
    "bool": {
      "must": [
        { "match": { "message": "error" } }
      ],
      "filter": [
        { "term": { "status": "500" } }
      ],
      "should": [
        { "match": { "service": "payment" } }
      ],
      "must_not": [
        { "term": { "user": "bot" } }
      ]
    }
  }
}
```

- MUST (score)
- FILTER (no score)
- SHOULD (boost)
- MUST_NOT (exclude)

4. RANGE QUERIES

```
{
  "query": {
    "range": {
      "timestamp": {
        "gte": "now-1h", "lte": "now"
      }
    }
  }
}
```

Use for:

- time ranges
- numbers
- dates
- sizes

5. WILDCARDS

```
{
  "query": {
    "wildcard": { "user": "adm*" }
  }
}
```

Useful for:

- prefix searches
- partial matches
- exploring data

6. AGGREGATIONS (BUCKETS & METRICS)

```
{
  "size": 0,
  "aggs": {
    "errors_by_service": {
      "terms": {
        "field": "service", "size": 10
      }
    }
  }
}
```



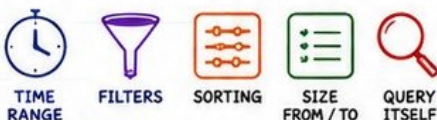
Group, count, average, min, max, percentiles, etc.

7. FILTERS VS QUERIES

FILTER CONTEXT	QUERY CONTEXT
• No scoring • Faster • Cached • Use for exact conditions	• Calculates score • Slower • Not cached • Use for relevance

8. SEARCH CONTEXT

Each search runs in a context that includes:



Different context = different results.

9. EXPENSIVE SEARCHES (AVOID)

- ✗ Leading wildcards (*term)
- ✗ Large result sets (high size)
- ✗ Expensive regex
- ✗ Script queries
- ✗ Deep pagination (from + size)
- ✗ High-cardinality aggregations

10. QUERY PROFILING CONCEPTS

- ✓ Use Profile API to see slow parts
- ✓ Identify slow queries & aggregations
- ✓ Check time spent per shard
- ✓ Optimize mappings & queries
- ✓ Measure before you change



PERFORMANCE TIP: FILTER CONTEXT CAN AVOID UNNECESSARY SCORING WORK. USE FILTERS FOR EXACT MATCH CONDITIONS WHEN POSSIBLE.

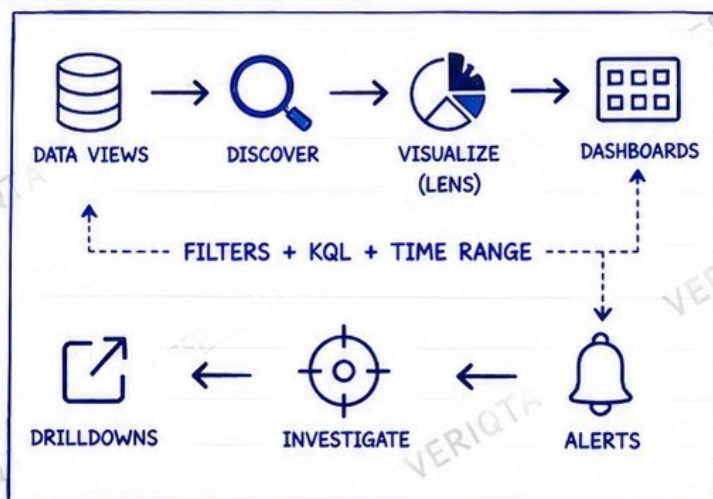
12. KIBANA FOR PRODUCTION OPERATIONS

TURN STORED TELEMETRY INTO OPERATIONAL INTELLIGENCE.

1. CORE BUILDING BLOCKS

-  **DATA VIEWS** - Connect to Elasticsearch data
-  **DISCOVER** - Explore and search raw data
-  **DASHBOARDS** - Visualize and monitor KPIs
-  **LENS** - Build charts & visualizations
-  **FILTERS** - Narrow down the data
- KQL** - Powerful query language
-  **TIME RANGES** - Focus on the right time
-  **SAVED SEARCHES** - Reuse and share queries
-  **DRILLDOWNS** - Jump deeper into details
-  **ALERTING CONCEPTS** - Detect issues early

2. KIBANA WORKFLOW



3. KQL EXAMPLES

- status: error # exact match
- service: "payment-api" # phrase match
- status: 500 and host: web-01 # boolean
- status: 4* # wildcard
- response_time > 1000 # range
- not status: 404 # exclude

4. DASHBOARD TYPES

OPERATIONS DASHBOARDS

- ☒ System health overview
- ☒ Infrastructure metrics
- ☒ Error rates & latency
- ☒ Capacity & saturation
- ☒ Service availability
- ☒ Top N breakdowns



SRE DASHBOARDS

- ☒ SLI / SLO monitoring
- ☒ Error budgets
- ☒ Burn rate
- ☒ Incident overview
- ☒ Service reliability
- ☒ Alert status



5. INCIDENT INVESTIGATION

-  **1. DETECT**
Alert fires or issue reported
-  **2. ORIENT**
Check dashboards & KPIs
-  **3. NARROW**
Use filters, KQL, time range
-  **4. DEEP DIVE**
Drilldowns & raw log analysis
-  **5. RESOLVE**
Validate fix & monitor

6. DASHBOARD ANTI-PATTERNS

- ✗ Too many panels (visual noise)
- ✗ No purpose or clear question
- ✗ Mixing metrics with no context
- ✗ Using defaults without thought
- ✗ Showing raw counts only
- ✗ No time range discipline
- ✗ Hard to read or cluttered
- ✗ Not actionable



VANITY METRICS

- Page views
- Total requests
- Success count
- Average CPU (no context)
- Top talkers (no reason)

7. BEST PRACTICES

- ☒ Start with the question
- ☒ Keep dashboards simple
- ☒ Use meaningful groupings
- ☒ Show rates, not just counts
- ☒ Add thresholds & context
- ☒ Use consistent time ranges
- ☒ Make it actionable
- ☒ Review and evolve



REAL WORLD LESSON

A DASHBOARD SHOULD ANSWER AN OPERATIONAL QUESTION, NOT JUST DISPLAY DATA.

13. ELASTICSEARCH PERFORMANCE ENGINEERING

UNDERSTAND WHY CLUSTERS BECOME SLOW.

1. HEAP PRESSURE & JVM MEMORY



HEAP LIMIT

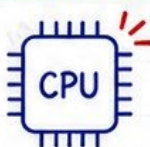
- Too much data in heap
- Large aggregations
- Fielddata / request cache
- Big terms queries
- Watch GC time & pauses

2. GARBAGE COLLECTION



- High GC time = slow nodes
- Long pauses impact searches
- Monitor GC in JVM stats
- Tune heap size, avoid spikes

3. CPU SATURATION



- High CPU from queries
- Scripted queries
- Aggregations
- Regex / wildcards
- Watch CPU per node

4. DISK I/O & PAGE CACHE



- Slow disk = slow everything
- Page cache holds index data
- More cache = faster searches
- Monitor I/O wait & cache hit

5. SEARCH LATENCY



- Depends on shard count
- Query complexity
- Field types & mappings
- Cache efficiency
- Track P50, P95, P99

6. INDEXING THROUGHPUT



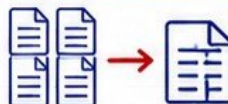
- Affected by refresh, replicas
- Bulk size & concurrency
- Disk speed & CPU
- Monitor docs indexed / sec

7. REFRESH INTERVAL



- Default 1s (near real-time)
- Lower = more CPU & I/O
- Higher = better indexing
- Tune based on use case

8. SEGMENT CREATION & MERGING



- Many small segments slow
- Merges consume I/O & CPU
- Monitor merge time
- Bigger segments = better

9. BULK INDEXING



- Use bulk API
- Right size (5-15 MB)
- Too small = overhead
- Too big = memory spikes
- Parallelism with care

10. SHARD SIZE MATTERS



- Too small = overhead
- Too big = slow recovery
- Aim for 20-50 GB
- Plan shards, not react

11. QUERY COMPLEXITY



- Wildcards (*), regex, scripts
- Deep pagination (from + size)
- Large result sets
- Use filters, not queries
- Profile slow queries

12. FIELD DATA RISKS



- Text fields in aggregations
- Loads data into heap
- Can cause OOM
- Use keyword or runtime fields

PERFORMANCE CHECKLIST

- ☒ Monitor JVM heap, GC, CPU, I/O
- ☒ Watch search & indexing latency
- ☒ Check shard size and count
- ☒ Analyze slow queries
- ☒ Review mappings & field types
- ☒ Tune refresh & bulk settings

WHAT TO MONITOR (KEY METRICS)



JVM HEAP
($< 75\%$)



GC TIME
($< 10\%$)



CPU USAGE
($< 70\%$)



I/O WAIT
($< 15\%$)



SEARCH LATENCY
(P95, P99)



INDEXING THROUGHPUT
(DOCS/SEC)



PERFORMANCE TIP: MORE NODES CANNOT COMPENSATE FOR FUNDAMENTALLY BAD INDEX DESIGN.

VERIQTA

ENGINEER • OBSERVE • UNDERSTAND

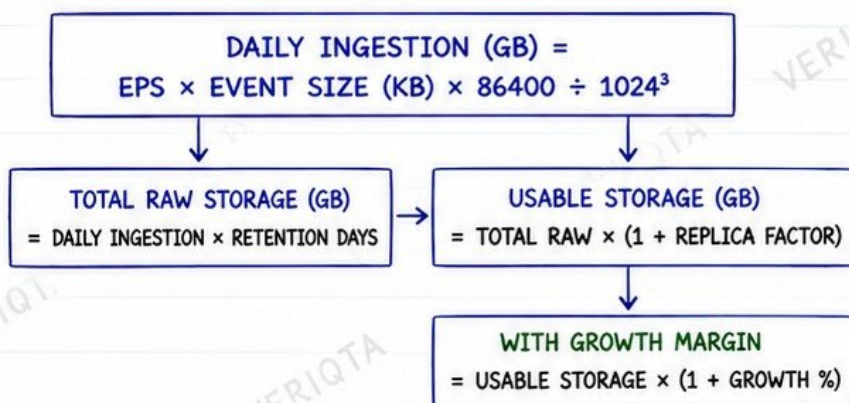
14. CAPACITY PLANNING AND SCALING ELK

PLAN BEFORE THE CLUSTER REACHES ITS LIMITS.

1. CAPACITY INPUTS

-  EVENTS PER SECOND (EPS)
-  AVERAGE EVENT SIZE (KB)
-  DAILY INGESTION VOLUME
-  RETENTION PERIOD (DAYS)
-  REPLICA OVERHEAD
-  GROWTH MARGIN (%)

2. STORAGE CALCULATION



3. HEAP SIZING CONSIDERATIONS



- Keep heap between 50–60% of total RAM
- Too small = GC pressure
- Too big = OS cache reduced
- Rule of thumb: Start with 31 GB heap per data node
- Monitor and tune



EXAMPLE

- EPS = 5,000
- AVG SIZE = 1 KB
- RETENTION = 30 DAYS
- REPLICA = 1 (1 replica)
- GROWTH = 30%

DAILY INGESTION ≈ 432 GB
USABLE (WITH REPLICA) ≈ 864 GB
WITH GROWTH 30% ≈ 1,123 GB

4. DISK WATERMARKS (DEFAULT)

-  LOW 85% – Cluster rebalances
-  HIGH 90% – Writes blocked to node
-  FLOOD STAGE 95% – Index read-only

5. SCALING APPROACH

SCALE DATA NODES



- Add nodes to increase storage & search capacity
- Rebalance shards
- Keep shard size healthy (20–50 GB)

SCALE INGESTION



- Add more Beats / Agents
- Use Kafka / Queue to buffer bursts
- Scale Logstash / Ingest nodes

SCALE LOGSTASH



- Add more workers
- Increase pipeline workers
- Use persistent queues
- Monitor CPU & memory



CAPACITY HEADROOM

- Keep 20–30% free storage
- Keep CPU < 70%
- Keep JVM heap < 75%
- Monitor trends, not just current values

6. HOTSPOT DETECTION

-  Uneven shard distribution
-  High CPU on specific nodes
-  Disk nearly full
-  High indexing or search latency
-  Thread pool rejections

7. GOOD PRACTICES

- ☒ Plan for peak ingestion, not daily average
- ☒ Use ILM to control retention and cost
- ☒ Right-size shards (20–50 GB)
- ☒ Keep mappings clean and controlled
- ☒ Test restores and scaling in lower environments
- ☒ Monitor everything and alert early

TARGET SHARD SIZE

- ☒ 20–50 GB → Ideal
- ☒ < 10 GB → Too small
- ☒ > 50 GB → Too large



SCALING NOTE: PLAN AROUND PEAK INGESTION, NOT DAILY AVERAGES.

15. SECURING THE ELK STACK

PROTECT ONE OF THE MOST SENSITIVE SYSTEMS
IN THE INFRASTRUCTURE.

1. TLS ENCRYPTION



- **NODE-TO-NODE ENCRYPTION**
Encrypt traffic between Elasticsearch nodes.
- **CLIENT ENCRYPTION**
Encrypt traffic from Clients, Beats, Logstash, and Kibana.

2. AUTHENTICATION



- Verify every user and service before access.
- Enable built-in authentication (native realm) or integrate with LDAP / SAML / OIDC.
- Disable anonymous access.

3. RBAC & LEAST PRIVILEGE



- Use Role-Based Access Control (RBAC).
- Grant only what is necessary.
- Follow the principle of least privilege.

4. API KEYS & SERVICE ACCOUNTS



- Use API keys for applications.
- Use service accounts for machine-to-machine access.
- Rotate keys regularly.

5. KIBANA ACCESS



- Protect Kibana with authentication.
- Restrict access to trusted users and networks.
- Use Spaces to separate teams and data.

6. INDEX-LEVEL PERMISSIONS



- Control which indices users can read or write.
- Prevent accidental or malicious data access.

7. FIELD & DOCUMENT RESTRICTIONS



- Hide sensitive fields (PII, tokens, passwords).
- Use field-level security (FLS) to restrict fields.
- Use document-level security (DLS) to restrict documents.

8. SECRET HANDLING



- Store credentials in secure keystores.
- Use environment variables or secret managers.
- Never hardcode passwords or tokens.

9. NETWORK SEGMENTATION



- Isolate ELK components in private networks.
- Restrict access using firewalls and security groups.
- Expose only necessary ports.

10. AUDIT LOGGING



- Enable audit logging for security events and access.
- Monitor authentication failures, role changes, and data access.
- Send audit logs to a separate secure index.

11. SENSITIVE LOG DATA



- Logs may contain secrets, tokens, credit cards, emails, IPs, and more.
- Mask or exclude sensitive data before ingestion.
- Apply retention and access controls.



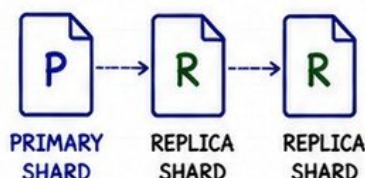
SECURITY REMINDER

LOGS FREQUENTLY CONTAIN CREDENTIALS, TOKENS, IDENTIFIERS, AND INTERNAL TOPOLOGY.

16. HIGH AVAILABILITY AND FAILURE RECOVERY

DESIGN ELK TO SURVIVE INFRASTRUCTURE FAILURES.

1. REPLICA STRATEGY



- Always keep replicas.
- Default: 1 replica (minimum).
- More replicas = higher availability.
- Place replicas on different nodes.

2. NODE AND MASTER FAILURE

NODE FAILURE



Replicas take over automatically.

MASTER FAILURE

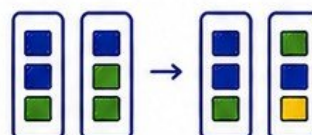


Another master-eligible node is elected.

3. CLUSTER HEALTH STATUS

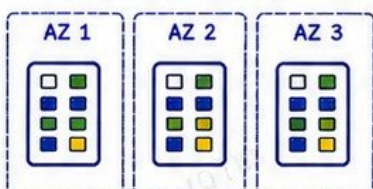


4. SHARD REALLOCATION



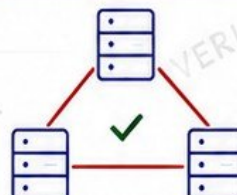
- Triggered by node loss, new nodes, or balancer.
- Elasticsearch moves shards to restore balance.
- Monitor reallocation for performance impact.

5. ALLOCATION AWARENESS & AVAILABILITY ZONES



- Spread shards across AZs.
- Prevents single AZ failure.
- Configure awareness attributes.
- Keeps primaries and replicas apart.

6. SPLIT-BRAIN HISTORY & QUORUM CONCEPTS



- Split-brain occurs when masters lose contact.
- Modern Elasticsearch uses quorum rules.
- Majority ($N/2 + 1$) of master-eligible nodes must agree to elect a master.
- Prevents two masters.

7. SNAPSHOT REPOSITORIES



- Snapshots are point-in-time backups.
- Store in S3, Azure Blob, GCS, NFS, or shared FS.
- Keep repository in a different location/account if possible.

8. BACKUP STRATEGY



- Define what to back up (indices, system, cluster state).
- Automate snapshot schedules.
- Verify snapshot success.
- Keep multiple generations.
- Store off-cluster.

9. RESTORE TESTING



- Test restores regularly.
- Restore in a lab or staging environment.
- Validate data completeness and integrity.
- Document the restore process.

10. DISASTER RECOVERY, RPO & RTO



DISASTER RECOVERY

Plan how to recover the entire cluster in a new environment.

RPO

How much data you can afford to lose.

RTO

How fast you must recover and be online.



PRODUCTION FAILURE: REPLICATION IS NOT A BACKUP.

REPLICAS PROTECT AGAINST NODE FAILURE.
SNAPSHOTS PROTECT AGAINST DATA LOSS.

17. MONITORING THE ELK STACK ITSELF

THE OBSERVABILITY PLATFORM ALSO NEEDS OBSERVABILITY.

WHAT TO MONITOR

1. **CLUSTER HEALTH**
Overall status: GREEN, YELLOW, RED
2. **NODE HEALTH**
Node up/down, roles, load
3. **JVM HEAP**
Heap usage %
4. **GC PRESSURE**
GC time %, pauses, collections
5. **CPU**
CPU usage % per node
6. **DISK**
Disk usage %, IOPS, throughput
7. **DISK WATERMARKS**
Low, High, Flood stage
8. **SEARCH LATENCY**
Query latency (p50, p95, p99)
9. **INDEXING LATENCY**
Indexing latency (p50, p95, p99)

ELASTICSEARCH METRICS

 THREAD POOLS Queue size; active, rejected	 REJECTED REQUESTS Rejected search and indexing	 SHARD COUNT Total shards in cluster
 UNASSIGNED SHARDS Shards not allocated	 SEARCH LATENCY Slow queries impact users	 INDEXING LATENCY Slow indexing builds backlog
 SEGMENT COUNT Too many segments slow search	 INDEX COUNT Total indices in cluster	 NODE STATS Load, fs, heap, threads, roles
 MERGE TIME High merge time = disk pressure	 FIELD DATA High fielddata uses heap	 SCRIPT COMPILATIONS High rate can cause spikes

LOGSTASH METRICS

- THROUGHPUT (EPS)
- PIPELINE QUEUE DEPTH
- WORKER UTILIZATION
- FILTER DURATION
- OUTPUT SUCCESS / FAILURE
- DEAD LETTER QUEUE SIZE
- JVM HEAP & GC

AGENT / BEATS METRICS

- AGENT ONLINE / OFFLINE
- EVENTS PER SECOND
- PUBLISHED VS ACKED
- HARVESTER STATUS
- REGISTRY STATE
- ERROR RATE
- OUTPUT LATENCY

ALERT THRESHOLDS (EXAMPLES)

- | | |
|--------------------------|----------|
| • CLUSTER HEALTH | != GREEN |
| • JVM HEAP USAGE | > 75% |
| • GC TIME | > 10% |
| • CPU USAGE | > 80% |
| • DISK WATERMARK HIGH | > 85% |
| • SEARCH LATENCY (p95) | > 1s |
| • INDEXING LATENCY (p95) | > 1s |
| • REJECTED REQUESTS RATE | > 1% |
| • UNASSIGNED SHARDS | > 0 |
| • LOGSTASH QUEUE DEPTH | > 20K |



SENIOR ENGINEER TIP:

MONITOR SATURATION SIGNALS BEFORE USER-FACING SYMPTOMS.
CATCH PRESSURE EARLY, PREVENT INCIDENTS.

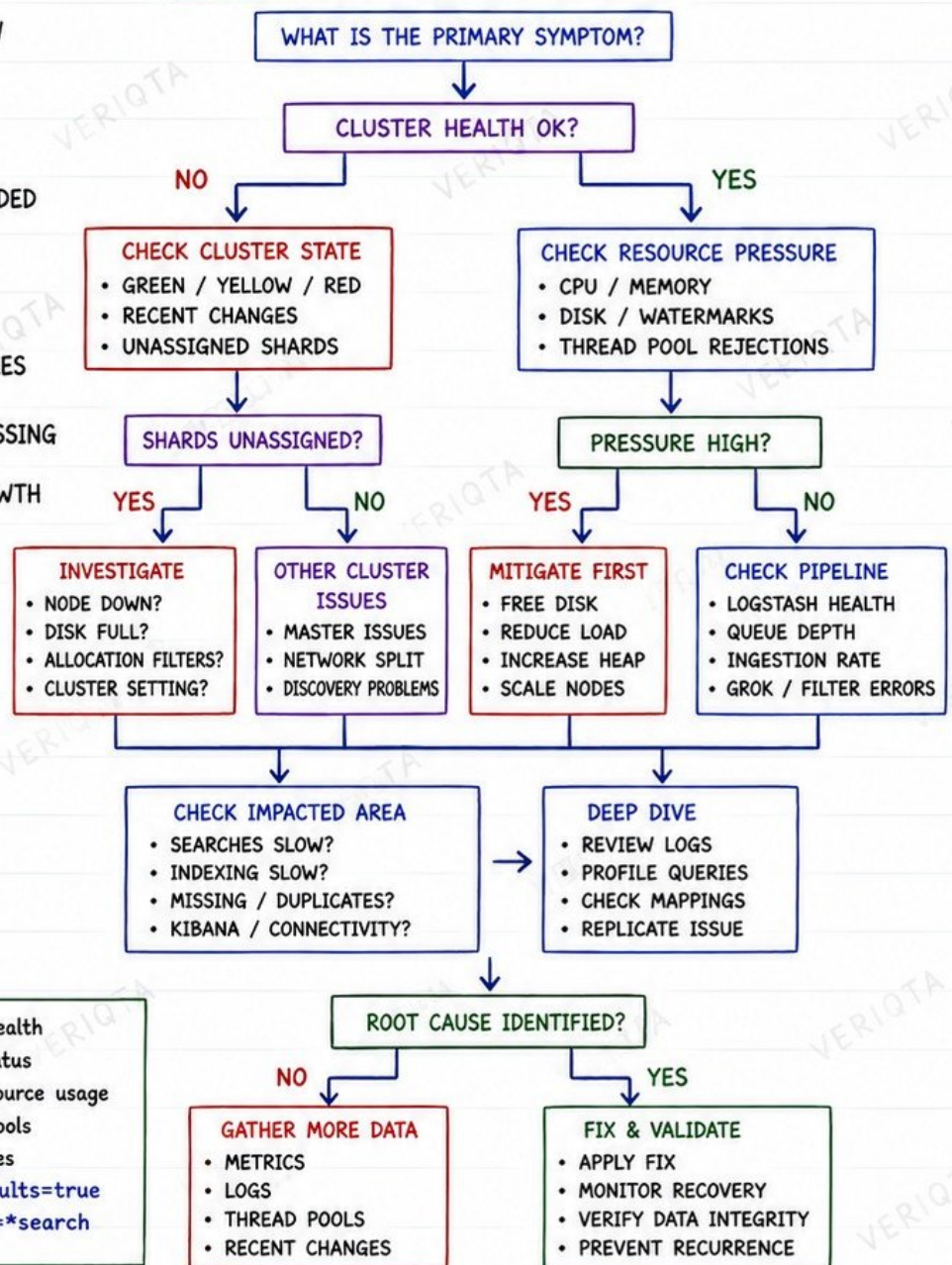
18. TROUBLESHOOTING COMMON ELK PRODUCTION FAILURES

USE SYMPTOMS TO NARROW THE FAILURE DOMAIN.

COMMON SYMPTOMS

1. CLUSTER TURNS YELLOW
2. CLUSTER TURNS RED
3. UNASSIGNED SHARDS
4. DISK WATERMARK EXCEEDED
5. JVM HEAP PRESSURE
6. SLOW SEARCHES
7. INDEXING LATENCY SPIKES
8. LOGSTASH STOPS PROCESSING
9. PERSISTENT QUEUE GROWTH
10. GROK FAILURES
11. MISSING LOGS
12. DUPLICATE EVENTS
13. KIBANA CANNOT REACH ELASTICSEARCH
14. MAPPING CONFLICTS
15. REJECTED REQUESTS

TROUBLESHOOTING DECISION TREE



QUICK CHECK COMMANDS

```

GET _cluster/health # Overall health
GET _cat/shards?v   # Shard status
GET _cat/nodes?v    # Node resource usage
GET _cat/thread_pool?v # Thread pools
GET _cat/indices?v  # Index sizes
GET _cluster/settings?include_defaults=true
GET _tasks?detailed=true&actions=*search
  
```



DEBUGGING INSIGHT:

START WITH CLUSTER STATE, RESOURCE PRESSURE, AND RECENT CHANGES. THESE EXPLAIN MOST ELK PRODUCTION INCIDENTS.

19. ELK RELIABILITY, COST, AND OPERATIONAL ANTI-PATTERNS

LEARN THE DECISIONS THAT QUIETLY CREATE EXPENSIVE SYSTEMS.

COMMON ANTI-PATTERNS

- 1**  **LOGGING EVERYTHING FOREVER**
 - Unlimited retention explodes cost.
 - Keep only what has business value.
- 2**  **EXCESSIVE DEBUG LOGGING**
 - Debug in production = noise + cost.
 - Use targeted, time-boxed debug.
- 3**  **OVERSHARDING**
 - Too many shards = heap pressure, slow cluster state, poor performance.
 - Target 20-50 GB per shard.
- 4**  **TINY INDICES**
 - Creates too many shards.
 - Inefficient use of resources.
- 5**  **HUGE SHARDS**
 - Slow recovery, long rebalances, risk of timeouts.
 - Keep shard size healthy.
- 6**  **MAPPING EXPLOSION**
 - Too many fields = high memory usage and unstable indices.
 - Control dynamic fields.
- 7**  **HIGH-CARDINALITY FIELDS**
 - User IDs, session IDs, IPs in agg cause high memory + slow queries.
 - Avoid aggregating on high-card fields.
- 8**  **UNBOUNDED RETENTION**
 - Old data rarely provides value.
 - Retention must follow business need.
- 9**  **EXPENSIVE WILDCARD SEARCHES**
 - Leading wildcards (*term) are very expensive.
 - Use exact fields and filters.
- 10**  **DASHBOARD OVERLOAD**
 - Too many dashboards = clutter.
 - Focus on actionable insights.
- 11**  **MISSING LIFECYCLE POLICIES**
 - No ILM = hot storage forever.
 - Always use ILM or data streams.
- 12**  **NO INGESTION BUFFERING**
 - Direct to ES = data loss on failures.
 - Use Logstash PQ or Kafka buffer.
- 13**  **NO CAPACITY HEADROOM**
 - Running at 90% = constant fire drills.
 - Plan for growth and peak.
- 14**  **TREATING ELASTICSEARCH AS THE SYSTEM OF RECORD**
 - ES is for search, not transactions.
 - Use the right system for the job.

COST VS OBSERVABILITY VALUE

COST DRIVERS

- Ingestion volume
- Storage & retention
- Shards & replicas
- Expensive queries
- Over-collection

OBSERVABILITY VALUE

- Faster incident response
- Better system reliability
- Security & compliance
- Business insights
- Customer experience

BALANCE COST WITH THE VALUE DELIVERED.
NOT ALL LOGS ARE CREATED EQUAL.



ENTERPRISE ADVICE

THE CHEAPEST EVENT
IS OFTEN THE ONE YOU
NEVER NEEDED TO INGEST.



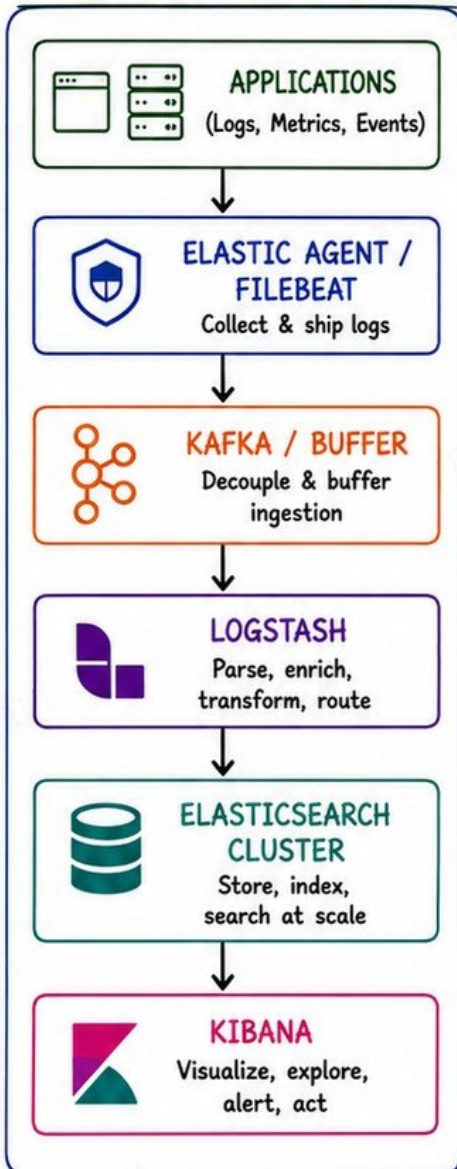
REMEMBER

GOOD LOGGING IS INTENTIONAL.
BAD LOGGING IS EXPENSIVE.

20. PRODUCTION ELK ARCHITECTURE AND INCIDENT WALKTHROUGH

FINISH WITH ONE REALISTIC SYSTEM RATHER THAN ANOTHER THEORY PAGE.

PRODUCTION ELK ARCHITECTURE



PRODUCTION INCIDENT WALKTHROUGH

1. INCIDENT TIMELINE (WHAT HAPPENED)

- 1  Log volume suddenly increases 5x
- 2  Logstash queue begins growing
- 3  Elasticsearch indexing latency rises
- 4  Heap pressure increases
- 5  Disk crosses the high watermark
- 6  Shards become unassigned
- 7  Kibana searches become slow
- 8  Alerts begin timing out

2. RESPONSE PLAYBOOK

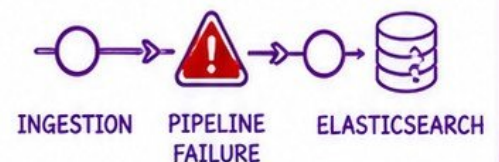
- 1  Identify the first bottleneck
- 2  Separate symptom from root cause
- 3  Stabilize ingestion
- 4  Protect Elasticsearch
- 5  Restore cluster health
- 6  Recover queued events

3. POST-RECOVERY ACTIONS

- 1  Validate data integrity
- 2  Prevent recurrence
- 3  Review & update alerts/thresholds
- 4  Improve runbooks & documentation

★ REAL WORLD LESSON

ELK INCIDENTS ARE USUALLY PIPELINE INCIDENTS, NOT ISOLATED ELASTICSEARCH INCIDENTS.



THE ELK JOURNEY: A CLEAR PROGRESSION

